CONFIGURATION ENGINE - ESPECIFICACIÓN TÉCNICA COMPLETA

SaidonClub v5.0 - Sistema de Switches Paramétricos

Fecha: 2026-04-22 | **Versión:** 1.0 | **Estado:** ESPECIFICACIÓN TÉCNICA

---

1. VISIÓN GENERAL

El Configuration Engine es el núcleo de SaidonClub v5.0. Permite al Super Admin controlar absolutamente todos los aspectos del sistema sin necesidad de modificar código ni hacer deploy.

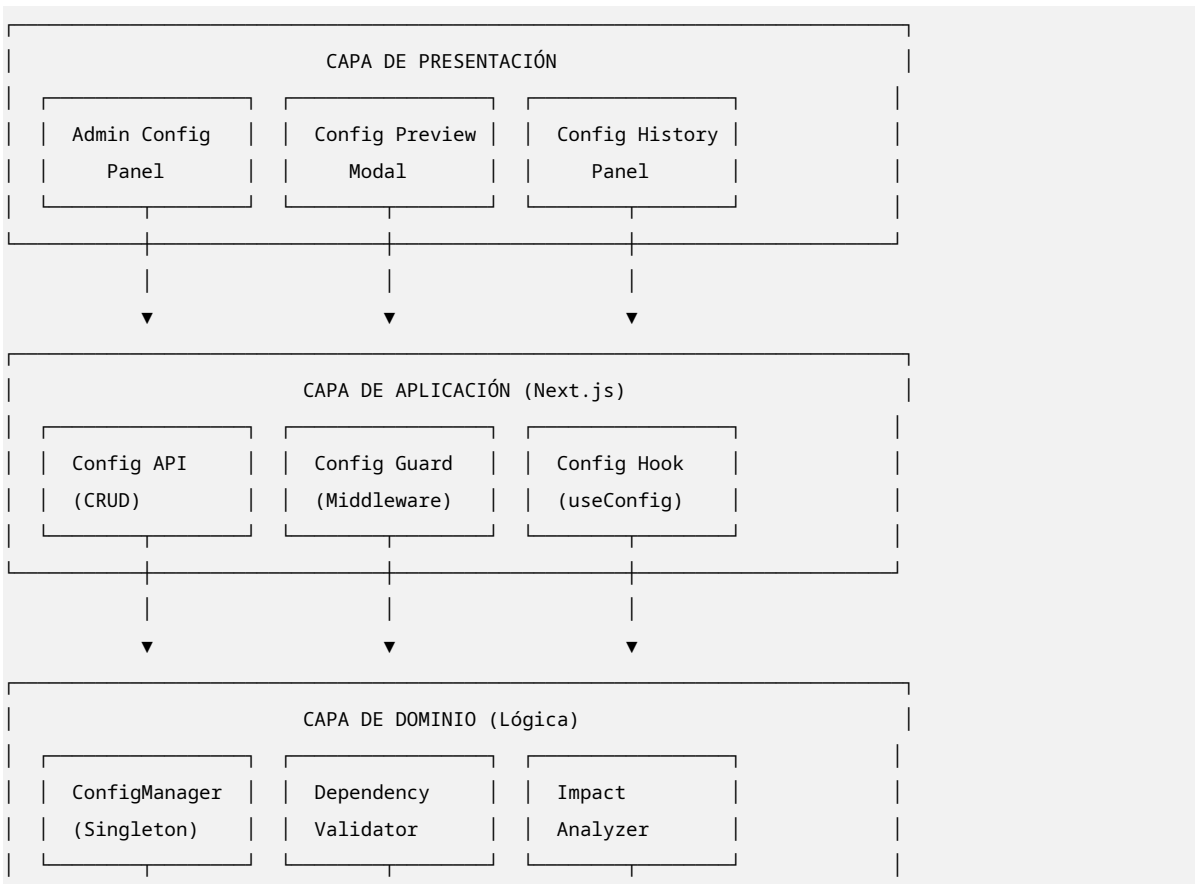
Objetivos:

1. **Parametrización Total:** Cada valor de negocio es configurable
2. **Seguridad:** Validación de dependencias y rangos antes de aplicar cambios
3. **Trazabilidad:** Historial completo de cambios con rollback
4. **Performance:** Cache distribuida con fallback a base de datos
5. **UX:** Panel intuitivo con preview de impacto

---

2. ARQUITECTURA DEL SISTEMA

2.1 Diagrama de Componentes



```
└──────────┬──────────┬──────────┬──────────┘
           │          │          │
           ▼          ▼          ▼
┌──────────────────────────────────────────────┐
│                CAPA DE DATOS                 │
│  ┌──────────┐  ┌──────────┐  ┌──────────┐    │
│  │SystemConfig│ │ConfigHistory│ │Upstash Redis│ │
│  │(Prisma/Neon)│ │(Audit Log)│ │(Cache)   │  │
│  └──────────┘  └──────────┘  └──────────┘    │
└──────────────────────────────────────────────┘
```

---

3. MODELO DE DATOS DETALLADO

3.1 Schema Prisma Completo

```
// ============================================
// CONFIGURATION ENGINE - MODELS
// ============================================

model SystemConfig {
  id              String          @id @default(uuid())
  key             String          @unique
  value           String
  type            ConfigType
  category        ConfigCategory
  description     String?
  editableBy      UserRole[]      @default([SUPER_ADMIN])
  requiresRestart Boolean         @default(false)
  dependencies    String[]        @default([])
  validationRegex String?
  minValue        Decimal?        @db.Decimal(15,4)
  maxValue        Decimal?        @db.Decimal(15,4)
  allowedValues   String[]        @default([])  // Para enums/arrays
  isActive        Boolean         @default(true)
  isPublic        Boolean         @default(false) // Si es visible en frontend
  displayOrder    Int             @default(0) @map("display_order")
  groupName       String?         @map("group_name")
  icon            String?
  createdAt       DateTime        @default(now()) @map("created_at")
  updatedAt       DateTime        @updatedAt @map("updated_at")
  updatedBy       String?         @map("updated_by")

  // Relaciones
  history         ConfigHistory[]

  @@index([category])
  @@index([isActive])
  @@index([key, isActive])
  @@map("system_config")
```

```
}

model ConfigHistory {
  id              String          @id @default(uuid())
  configId        String          @map("config_id")
  key             String
  oldValue        String          @map("old_value")
  newValue        String          @map("new_value")
  changedBy       String          @map("changed_by")
  changedByEmail  String          @map("changed_by_email")
  reason          String?
  ipAddress       String?         @map("ip_address")
  userAgent       String?         @map("user_agent")
  createdAt       DateTime        @default(now()) @map("created_at")

  config          SystemConfig    @relation(fields: [configId], references: [id], onDelete: Cascade)

  @@index([configId])
  @@index([createdAt])
  @@map("config_history")
}

enum ConfigType {
  BOOLEAN
  NUMBER
  DECIMAL
  STRING
  JSON
  ARRAY
  CRON
  EMAIL
  URL
  COLOR
}

enum ConfigCategory {
  GENERAL
  AUTH
  MARKETPLACE
  MLM
  MEMBERSHIP
  WALLET
  SERVICES
  UI
  SECURITY
  NOTIFICATIONS
  PAYMENTS
  SHIPPING
  TAXES
}
```

3.2 Seeds Iniciales (Obligatorios)

```
// prisma/seeds/config.ts
export const defaultConfigs = [
  // === GENERAL ===
  {
    key: 'app_name',
    value: 'SaidonClub',
    type: 'STRING',
    category: 'GENERAL',
    description: 'Nombre de la aplicación',
    editableBy: ['SUPER_ADMIN'],
    isPublic: true,
  },
  {
    key: 'app_maintenance_mode',
    value: 'false',
    type: 'BOOLEAN',
    category: 'GENERAL',
    description: 'Modo mantenimiento (bloquea acceso público)',
    editableBy: ['SUPER_ADMIN'],
  },

  // === AUTH ===
  {
    key: 'auth_sponsor_required',
    value: 'true',
    type: 'BOOLEAN',
    category: 'AUTH',
    description: 'Requerir patrocinador obligatorio para registro',
    editableBy: ['SUPER_ADMIN'],
  },
  {
    key: 'auth_google_oauth_enabled',
    value: 'true',
    type: 'BOOLEAN',
    category: 'AUTH',
    description: 'Permitir login con Google',
    editableBy: ['SUPER_ADMIN'],
  },
  {
    key: 'auth_session_timeout_hours',
    value: '24',
    type: 'NUMBER',
    category: 'AUTH',
    description: 'Duración de sesión en horas',
    editableBy: ['SUPER_ADMIN'],
    minValue: 1,
    maxValue: 720,
  },
```

```
// === MEMBERSHIP ===
{
  key: 'membership_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MEMBERSHIP',
  description: 'Activar sistema de membresías',
  editableBy: ['SUPER_ADMIN'],
},
{
  key: 'membership_include_products',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MEMBERSHIP',
  description: 'Incluir productos físicos con la membresía',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['membership_enabled'],
},
{
  key: 'membership_preferente_price',
  value: '39.00',
  type: 'DECIMAL',
  category: 'MEMBERSHIP',
  description: 'Precio membresía Preferente (USD)',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 1000,
  dependencies: ['membership_enabled'],
},
{
  key: 'membership_pionero_price',
  value: '129.00',
  type: 'DECIMAL',
  category: 'MEMBERSHIP',
  description: 'Precio membresía Pionero (USD)',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 1000,
  dependencies: ['membership_enabled'],
},
{
  key: 'membership_preferente_product_value',
  value: '10.00',
  type: 'DECIMAL',
  category: 'MEMBERSHIP',
  description: 'Valor en productos incluidos con Preferente',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['membership_enabled', 'membership_include_products'],
},
```

```
{
  key: 'membership_pionero_product_value',
  value: '30.00',
  type: 'DECIMAL',
  category: 'MEMBERSHIP',
  description: 'Valor en productos incluidos con Pionero',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 200,
  dependencies: ['membership_enabled', 'membership_include_products'],
},
{
  key: 'membership_upgrade_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MEMBERSHIP',
  description: 'Permitir upgrades de membresía',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['membership_enabled'],
},
{
  key: 'membership_city_discounts_enabled',
  value: 'false',
  type: 'BOOLEAN',
  category: 'MEMBERSHIP',
  description: 'Activar descuentos por ciudad',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['membership_enabled'],
},
{
  key: 'membership_auto_activate_days',
  value: '30',
  type: 'NUMBER',
  category: 'MEMBERSHIP',
  description: 'Días de activación inicial post-compra',
  editableBy: ['SUPER_ADMIN'],
  minValue: 1,
  maxValue: 365,
},

// === MLM - GENERAL ===
{
  key: 'mlm_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MLM',
  description: 'Activar motor MLM completo',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['membership_enabled', 'wallet_enabled'],
},
{
```

```
    key: 'mlm_royalty_enabled',
    value: 'true',
    type: 'BOOLEAN',
    category: 'MLM',
    description: 'Activar sistema de regalías',
    editableBy: ['SUPER_ADMIN'],
    dependencies: ['mlm_enabled'],
  },
  {
    key: 'mlm_royalty_percentage',
    value: '50.00',
    type: 'DECIMAL',
    category: 'MLM',
    description: 'Porcentaje del margen para regalías',
    editableBy: ['SUPER_ADMIN'],
    minValue: 0,
    maxValue: 100,
    dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
  },
  {
    key: 'mlm_royalty_levels',
    value: '8',
    type: 'NUMBER',
    category: 'MLM',
    description: 'Niveles de profundidad para regalías',
    editableBy: ['SUPER_ADMIN'],
    minValue: 1,
    maxValue: 20,
    dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
  },
  {
    key: 'mlm_royalty_l1_pct',
    value: '15.00',
    type: 'DECIMAL',
    category: 'MLM',
    description: 'Porcentaje regalía Nivel 1',
    editableBy: ['SUPER_ADMIN'],
    minValue: 0,
    maxValue: 100,
    dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
  },
  {
    key: 'mlm_royalty_l2_pct',
    value: '10.00',
    type: 'DECIMAL',
    category: 'MLM',
    description: 'Porcentaje regalía Nivel 2',
    editableBy: ['SUPER_ADMIN'],
    minValue: 0,
    maxValue: 100,
    dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
```

```
},
{
  key: 'mlm_royalty_l3_pct',
  value: '10.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Porcentaje regalía Nivel 3',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
},
{
  key: 'mlm_royalty_l4_pct',
  value: '5.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Porcentaje regalía Nivel 4',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
},
{
  key: 'mlm_royalty_l5_pct',
  value: '4.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Porcentaje regalía Nivel 5',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
},
{
  key: 'mlm_royalty_l6_pct',
  value: '3.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Porcentaje regalía Nivel 6',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
},
{
  key: 'mlm_royalty_l7_pct',
  value: '2.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Porcentaje regalía Nivel 7',
```

```
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
},
{
  key: 'mlm_royalty_l8_pct',
  value: '1.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Porcentaje regalía Nivel 8',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['mlm_enabled', 'mlm_royalty_enabled'],
},

// === MLM - SEED BONUS ===
{
  key: 'mlm_seed_bonus_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MLM',
  description: 'Activar bono semilla',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['mlm_enabled'],
},
{
  key: 'mlm_seed_preferente_n1',
  value: '10.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono semilla Preferente Nivel 1',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100,
  dependencies: ['mlm_enabled', 'mlm_seed_bonus_enabled'],
},
{
  key: 'mlm_seed_pionero_n1',
  value: '43.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono semilla Pionero Nivel 1',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 200,
  dependencies: ['mlm_enabled', 'mlm_seed_bonus_enabled'],
},
{
  key: 'mlm_seed_pionero_n2_8',
```

```
  value: '1.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono semilla Pionero Niveles 2-8',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 50,
  dependencies: ['mlm_enabled', 'mlm_seed_bonus_enabled'],
},
{
  key: 'mlm_seed_upgrade_n1',
  value: '33.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono upgrade Nivel 1',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 200,
  dependencies: ['mlm_enabled', 'mlm_seed_bonus_enabled'],
},
{
  key: 'mlm_seed_upgrade_n2_8',
  value: '1.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono upgrade Niveles 2-8',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 50,
  dependencies: ['mlm_enabled', 'mlm_seed_bonus_enabled'],
},

// === MLM - RANKS ===
{
  key: 'mlm_ranks_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MLM',
  description: 'Activar sistema de rangos',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['mlm_enabled'],
},
{
  key: 'mlm_rank_35_rule_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MLM',
  description: 'Activar regla 35% de concentración',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
```

```
{
  key: 'mlm_rank_35_base_points',
  value: '2500.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos base para cálculo del 35% (Plata)',
  editableBy: ['SUPER_ADMIN'],
  minValue: 100,
  maxValue: 10000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled', 'mlm_rank_35_rule_enabled'],
},
{
  key: 'mlm_fidelity_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MLM',
  description: 'Activar bono fidelidad',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},

// === MLM - ACTIVATION ===
{
  key: 'mlm_activation_rolling_days',
  value: '30',
  type: 'NUMBER',
  category: 'MLM',
  description: 'Días de ventana rolling para activación',
  editableBy: ['SUPER_ADMIN'],
  minValue: 1,
  maxValue: 365,
  dependencies: ['mlm_enabled'],
},
{
  key: 'mlm_activation_min_points',
  value: '50.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos mínimos para activación por consumo',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000,
  dependencies: ['mlm_enabled'],
},

// === RANKS - CONFIGURABLE ===
{
  key: 'rank_plata_points',
  value: '2500.00',
  type: 'DECIMAL',
  category: 'MLM',
```

```
  description: 'Puntos requeridos para rango Plata',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_plata_bonus',
  value: '200.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono mensual rango Plata',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_oro_points',
  value: '7000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos requeridos para rango Oro',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_oro_bonus',
  value: '400.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono mensual rango Oro',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_zafiro_points',
  value: '20000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos requeridos para rango Zafiro',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
```

```
  key: 'rank_zafiro_bonus',
  value: '900.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono mensual rango Zafiro',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_esmeralda_points',
  value: '55000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos requeridos para rango Esmeralda',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_esmeralda_bonus',
  value: '2000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono mensual rango Esmeralda',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_rubi_points',
  value: '155000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos requeridos para rango Rubí',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_rubi_bonus',
  value: '6000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono mensual rango Rubí',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
```

```
  maxValue: 100000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_diamante_points',
  value: '450000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos requeridos para rango Diamante',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_diamante_bonus',
  value: '20000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono mensual rango Diamante',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_diamante_azul_points',
  value: '1200000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Puntos requeridos para rango Diamante Azul',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},
{
  key: 'rank_diamante_azul_bonus',
  value: '50000.00',
  type: 'DECIMAL',
  category: 'MLM',
  description: 'Bono mensual rango Diamante Azul',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 100000,
  dependencies: ['mlm_enabled', 'mlm_ranks_enabled'],
},

// === WALLET ===
{
  key: 'wallet_enabled',
```

```
  value: 'true',
  type: 'BOOLEAN',
  category: 'WALLET',
  description: 'Activar sistema de wallet',
  editableBy: ['SUPER_ADMIN'],
},
{
  key: 'wallet_withdrawal_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'WALLET',
  description: 'Permitir retiros de wallet',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['wallet_enabled'],
},
{
  key: 'wallet_withdrawal_min',
  value: '50.00',
  type: 'DECIMAL',
  category: 'WALLET',
  description: 'Monto mínimo para retiro',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000,
  dependencies: ['wallet_enabled', 'wallet_withdrawal_enabled'],
},
{
  key: 'wallet_withdrawal_fee_pct',
  value: '2.00',
  type: 'DECIMAL',
  category: 'WALLET',
  description: 'Comisión por retiro (%)',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 50,
  dependencies: ['wallet_enabled', 'wallet_withdrawal_enabled'],
},
{
  key: 'wallet_points_transfer_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'WALLET',
  description: 'Permitir transferencia de puntos entre usuarios',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['wallet_enabled'],
},
{
  key: 'wallet_points_redemption_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'WALLET',
```

```
    description: 'Permitir canje de puntos por dinero',
    editableBy: ['SUPER_ADMIN'],
    dependencies: ['wallet_enabled'],
  },
  {
    key: 'wallet_points_redemption_rate',
    value: '1.00',
    type: 'DECIMAL',
    category: 'WALLET',
    description: 'Tasa de canje: 1 punto = $X',
    editableBy: ['SUPER_ADMIN'],
    minValue: 0.01,
    maxValue: 100,
    dependencies: ['wallet_enabled', 'wallet_points_redemption_enabled'],
  },
  {
    key: 'wallet_points_redemption_day',
    value: 'friday',
    type: 'STRING',
    category: 'WALLET',
    description: 'Día de la semana para canje de puntos',
    editableBy: ['SUPER_ADMIN'],
    allowedValues: ['monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday'],
    dependencies: ['wallet_enabled', 'wallet_points_redemption_enabled'],
  },
  {
    key: 'wallet_debt_auto_deduct',
    value: 'true',
    type: 'BOOLEAN',
    category: 'WALLET',
    description: 'Deducción automática de deudas de devoluciones',
    editableBy: ['SUPER_ADMIN'],
    dependencies: ['wallet_enabled'],
  },

  // === MARKETPLACE ===
  {
    key: 'marketplace_enabled',
    value: 'true',
    type: 'BOOLEAN',
    category: 'MARKETPLACE',
    description: 'Activar marketplace de productos',
    editableBy: ['SUPER_ADMIN'],
  },
  {
    key: 'marketplace_guest_checkout',
    value: 'true',
    type: 'BOOLEAN',
    category: 'MARKETPLACE',
    description: 'Permitir compra sin registro',
    editableBy: ['SUPER_ADMIN'],
```

```
  dependencies: ['marketplace_enabled'],
},
{
  key: 'marketplace_guest_cookie_hours',
  value: '24',
  type: 'NUMBER',
  category: 'MARKETPLACE',
  description: 'Horas de duración cookie de invitado',
  editableBy: ['SUPER_ADMIN'],
  minValue: 1,
  maxValue: 720,
  dependencies: ['marketplace_enabled', 'marketplace_guest_checkout'],
},
{
  key: 'marketplace_shipping_free_threshold',
  value: '100.00',
  type: 'DECIMAL',
  category: 'MARKETPLACE',
  description: 'Monto mínimo para envío gratis',
  editableBy: ['SUPER_ADMIN'],
  minValue: 0,
  maxValue: 10000,
  dependencies: ['marketplace_enabled'],
},
{
  key: 'marketplace_points_checkout_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MARKETPLACE',
  description: 'Permitir pagar con puntos en checkout',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['marketplace_enabled', 'wallet_enabled'],
},
{
  key: 'marketplace_wallet_checkout_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MARKETPLACE',
  description: 'Permitir pagar con wallet en checkout',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['marketplace_enabled', 'wallet_enabled'],
},
{
  key: 'marketplace_stripe_checkout_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MARKETPLACE',
  description: 'Permitir pagar con Stripe en checkout',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['marketplace_enabled'],
},
```

```
{
  key: 'marketplace_price_validation_strict',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MARKETPLACE',
  description: 'Validación server-side estricta de precios',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['marketplace_enabled'],
},
{
  key: 'marketplace_reviews_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MARKETPLACE',
  description: 'Activar sistema de reviews',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['marketplace_enabled'],
},
{
  key: 'marketplace_wishlist_enabled',
  value: 'false',
  type: 'BOOLEAN',
  category: 'MARKETPLACE',
  description: 'Activar lista de deseos',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['marketplace_enabled'],
},

// === SERVICES ===
{
  key: 'services_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'SERVICES',
  description: 'Activar marketplace de servicios profesionales',
  editableBy: ['SUPER_ADMIN'],
},
{
  key: 'services_qr_validation_enabled',
  value: 'true',
  type: 'BOOLEAN',
  category: 'SERVICES',
  description: 'Activar validación QR dinámico',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['services_enabled'],
},
{
  key: 'services_platform_fee_pct',
  value: '5.00',
  type: 'DECIMAL',
  category: 'SERVICES',
```

```
    description: 'Porcentaje de retención por servicio',
    editableBy: ['SUPER_ADMIN'],
    minValue: 0,
    maxValue: 100,
    dependencies: ['services_enabled'],
  },
  {
    key: 'services_escrow_enabled',
    value: 'true',
    type: 'BOOLEAN',
    category: 'SERVICES',
    description: 'Activar escrow para servicios',
    editableBy: ['SUPER_ADMIN'],
    dependencies: ['services_enabled'],
  },
  {
    key: 'services_geolocation_enabled',
    value: 'true',
    type: 'BOOLEAN',
    category: 'SERVICES',
    description: 'Activar filtro por geolocalización',
    editableBy: ['SUPER_ADMIN'],
    dependencies: ['services_enabled'],
  },
  {
    key: 'services_calendar_enabled',
    value: 'true',
    type: 'BOOLEAN',
    category: 'SERVICES',
    description: 'Activar calendario de disponibilidad',
    editableBy: ['SUPER_ADMIN'],
    dependencies: ['services_enabled'],
  },

  // === CLOSURE ===
  {
    key: 'closure_enabled',
    value: 'true',
    type: 'BOOLEAN',
    category: 'MLM',
    description: 'Activar cierre semanal automático',
    editableBy: ['SUPER_ADMIN'],
    dependencies: ['mlm_enabled'],
  },
  {
    key: 'closure_day',
    value: 'friday',
    type: 'STRING',
    category: 'MLM',
    description: 'Día de la semana para cierre',
    editableBy: ['SUPER_ADMIN'],
```

```
  allowedValues: ['monday', 'tuesday', 'wednesday', 'thursday', 'friday', 'saturday', 'sunday'],
  dependencies: ['mlm_enabled', 'closure_enabled'],
},
{
  key: 'closure_time',
  value: '17:00',
  type: 'STRING',
  category: 'MLM',
  description: 'Hora del cierre (HH:MM, Ecuador)',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['mlm_enabled', 'closure_enabled'],
},
{
  key: 'closure_detection_hours',
  value: '12',
  type: 'NUMBER',
  category: 'MLM',
  description: 'Horas de fase de detección',
  editableBy: ['SUPER_ADMIN'],
  minValue: 1,
  maxValue: 72,
  dependencies: ['mlm_enabled', 'closure_enabled'],
},
{
  key: 'closure_manual_override',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MLM',
  description: 'Permitir a Super Admin forzar cierre manual',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['mlm_enabled', 'closure_enabled'],
},
{
  key: 'closure_auto_rollback',
  value: 'true',
  type: 'BOOLEAN',
  category: 'MLM',
  description: 'Rollback automático si falla el cierre',
  editableBy: ['SUPER_ADMIN'],
  dependencies: ['mlm_enabled', 'closure_enabled'],
},

// === SECURITY ===
{
  key: 'security_rate_limit_login',
  value: '5',
  type: 'NUMBER',
  category: 'SECURITY',
  description: 'Intentos de login antes de bloqueo',
  editableBy: ['SUPER_ADMIN'],
  minValue: 1,
```

```
    maxValue: 100,
  },
  {
    key: 'security_rate_limit_window',
    value: '15',
    type: 'NUMBER',
    category: 'SECURITY',
    description: 'Minutos de ventana para rate limit',
    editableBy: ['SUPER_ADMIN'],
    minValue: 1,
    maxValue: 1440,
  },
  {
    key: 'security_2fa_enabled',
    value: 'false',
    type: 'BOOLEAN',
    category: 'SECURITY',
    description: 'Requerir 2FA para roles admin',
    editableBy: ['SUPER_ADMIN'],
  },
  {
    key: 'security_audit_log_all',
    value: 'true',
    type: 'BOOLEAN',
    category: 'SECURITY',
    description: 'Loggear todas las acciones financieras',
    editableBy: ['SUPER_ADMIN'],
  },
];
```

---

4. IMPLEMENTACIÓN TÉCNICA

4.1 ConfigManager (Singleton)

```
// lib/config.ts
import { Redis } from '@upstash/redis';
import { prisma } from './prisma';
import { EventLogService } from './event-log';

const redis = new Redis({
  url: process.env.UPSTASH_REDIS_REST_URL!,
  token: process.env.UPSTASH_REDIS_REST_TOKEN!,
});

const CACHE_TTL = 60; // segundos
const CACHE_PREFIX = 'config:';

class ConfigManager {
  private localCache: Map<string, { value: any; expires: number }> = new Map();
```

```
/**
 * Obtiene una configuración por su clave
 * Estrategia: Local Cache → Redis → DB
 */
async get<T>(key: string, defaultValue?: T): Promise<T> {
  // 1. Verificar cache local (memoria)
  const local = this.localCache.get(key);
  if (local && local.expires > Date.now()) {
    return local.value as T;
  }

  // 2. Verificar Redis
  try {
    const cached = await redis.get(`${CACHE_PREFIX}${key}`);
    if (cached !== null) {
      const parsed = this.parseValue(cached as string, this.inferType(cached));
      this.setLocalCache(key, parsed);
      return parsed as T;
    }
  } catch (error) {
    console.warn(`Redis fallback para config ${key}:`, error);
  }

  // 3. Fallback a base de datos
  const config = await prisma.systemConfig.findUnique({
    where: { key, isActive: true },
  });

  if (config) {
    const value = this.parseValue(config.value, config.type);

    // Guardar en Redis (fire and forget)
    redis.setex(`${CACHE_PREFIX}${key}`, CACHE_TTL, JSON.stringify(value))
      .catch(err => console.warn(`Error guardando en Redis ${key}:`, err));

    this.setLocalCache(key, value);
    return value;
  }

  // 4. Valor por defecto
  if (defaultValue !== undefined) {
    return defaultValue;
  }

  throw new Error(`Configuración '${key}' no encontrada y no tiene valor por defecto`);
}

/**
 * Establece una configuración con validaciones completas
 */
```

```typescript
async set(key: string, value: any, userId: string, reason?: string): Promise<void> {
  // 1. Obtener configuración actual
  const config = await prisma.systemConfig.findUnique({ where: { key } });
  if (!config) {
    throw new Error(`Configuración '${key}' no existe`);
  }

  // 2. Validar permisos de rol
  const user = await prisma.user.findUnique({ where: { id: userId } });
  if (!user || !config.editableBy.includes(user.role)) {
    throw new Error(`No tienes permiso para modificar '${key}'`);
  }

  // 3. Validar tipo
  const stringValue = String(value);
  if (!this.validateType(stringValue, config.type)) {
    throw new Error(`Valor inválido para tipo ${config.type}`);
  }

  // 4. Validar regex
  if (config.validationRegex && !new RegExp(config.validationRegex).test(stringValue)) {
    throw new Error(`Valor no cumple con el formato requerido`);
  }

  // 5. Validar rango
  if (config.minValue !== null || config.maxValue !== null) {
    const numValue = Number(value);
    if (config.minValue !== null && numValue < Number(config.minValue)) {
      throw new Error(`Valor mínimo permitido: ${config.minValue}`);
    }
    if (config.maxValue !== null && numValue > Number(config.maxValue)) {
      throw new Error(`Valor máximo permitido: ${config.maxValue}`);
    }
  }

  // 6. Validar valores permitidos
  if (config.allowedValues.length > 0 && !config.allowedValues.includes(stringValue)) {
    throw new Error(`Valor debe ser uno de: ${config.allowedValues.join(', ')}`);
  }

  // 7. Validar dependencias
  await this.validateDependencies(key, value);

  // 8. Guardar historial
  await prisma.configHistory.create({
    data: {
      configId: config.id,
      key,
      oldValue: config.value,
      newValue: stringValue,
      changedBy: userId,
```

```
      changedByEmail: user.email,
      reason: reason || 'Cambio desde panel admin',
    },
  });

  // 9. Actualizar configuración
  await prisma.systemConfig.update({
    where: { key },
    data: { value: stringValue, updatedBy: userId },
  });

  // 10. Invalidar caches
  await this.invalidateCache(key);

  // 11. Loggear evento
  await EventLogService.log({
    aggregateType: 'SYSTEM_CONFIG',
    eventType: 'CONFIG_CHANGED',
    userId,
    payload: { key, oldValue: config.value, newValue: stringValue, reason },
  });
}

/**
 * Valida que las dependencias sean consistentes
 */
private async validateDependencies(key: string, value: any): Promise<void> {
  const config = await prisma.systemConfig.findUnique({ where: { key } });
  if (!config) return;

  // Si estamos desactivando, verificar que no sea dependencia de otros activos
  if (value === false || value === 'false') {
    const dependents = await prisma.systemConfig.findMany({
      where: {
        dependencies: { has: key },
        isActive: true,
        value: 'true',
      },
    });

    if (dependents.length > 0) {
      const names = dependents.map(d => `${d.key} (${d.description})`).join(', ');
      throw new Error(
        `No puedes desactivar '${key}' porque es requerido por: ${names}. ` +
        `Desactiva esos módulos primero.`
      );
    }
  }

  // Si estamos activando, verificar que las dependencias estén activas
  if (value === true || value === 'true') {
```

```
    for (const depKey of config.dependencies) {
      const depConfig = await prisma.systemConfig.findUnique({
        where: { key: depKey },
      });
      if (!depConfig || depConfig.value !== 'true') {
        throw new Error(
          `No puedes activar '${key}' porque requiere que '${depKey}' esté activo. ` +
          `Actívalo primero.`
        );
      }
    }
  }
}

/**
 * Invalida todas las capas de cache para una clave
 */
async invalidateCache(key: string): Promise<void> {
  this.localCache.delete(key);
  try {
    await redis.del(`${CACHE_PREFIX}${key}`);
  } catch (error) {
    console.warn(`Error invalidando Redis ${key}:`, error);
  }
}

/**
 * Invalida todas las configuraciones (usar con cuidado)
 */
async invalidateAllCache(): Promise<void> {
  this.localCache.clear();
  try {
    const keys = await redis.keys(`${CACHE_PREFIX}*`);
    if (keys.length > 0) {
      await redis.del(...keys);
    }
  } catch (error) {
    console.warn('Error invalidando cache global:', error);
  }
}

/**
 * Obtiene múltiples configuraciones en una sola llamada
 */
async getMany(keys: string[]): Promise<Record<string, any>> {
  const result: Record<string, any> = {};
  await Promise.all(
    keys.map(async (key) => {
      try {
        result[key] = await this.get(key);
      } catch (error) {
```

```
        result[key] = null;
      }
    })
  );
  return result;
}

/**
 * Rollback a una versión anterior
 */
async rollback(key: string, historyId: string, userId: string): Promise<void> {
  const history = await prisma.configHistory.findUnique({
    where: { id: historyId },
  });

  if (!history || history.key !== key) {
    throw new Error('Historial no encontrado');
  }

  await this.set(key, history.oldValue, userId, `Rollback a versión de ${history.createdAt}`);
}

// Métodos privados helpers...
private parseValue(value: string, type: string): any {
  switch (type) {
    case 'BOOLEAN': return value === 'true';
    case 'NUMBER': return Number(value);
    case 'DECIMAL': return Number(value);
    case 'JSON': return JSON.parse(value);
    case 'ARRAY': return JSON.parse(value);
    default: return value;
  }
}

private inferType(value: any): string {
  if (typeof value === 'boolean') return 'BOOLEAN';
  if (typeof value === 'number') return 'NUMBER';
  if (Array.isArray(value)) return 'ARRAY';
  if (typeof value === 'object') return 'JSON';
  return 'STRING';
}

private validateType(value: string, type: string): boolean {
  switch (type) {
    case 'BOOLEAN': return ['true', 'false'].includes(value);
    case 'NUMBER': return !isNaN(Number(value)) && Number.isInteger(Number(value));
    case 'DECIMAL': return !isNaN(Number(value));
    case 'JSON':
      try { JSON.parse(value); return true; } catch { return false; }
    case 'ARRAY':
      try { return Array.isArray(JSON.parse(value)); } catch { return false; }
```

```
      default: return true;
    }
  }

  private setLocalCache(key: string, value: any): void {
    this.localCache.set(key, {
      value,
      expires: Date.now() + (CACHE_TTL * 1000),
    });
  }
}

export const config = new ConfigManager();
```

4.2 Hook React para Configuraciones

```
// hooks/use-config.ts
'use client';

import { useQuery } from '@tanstack/react-query';

export function useConfig<T>(key: string, defaultValue?: T) {
  return useQuery({
    queryKey: ['config', key],
    queryFn: async () => {
      const res = await fetch(`/api/admin/config/${key}`);
      if (!res.ok) return defaultValue;
      return res.json() as Promise<T>;
    },
    staleTime: 30000, // 30 segundos
    refetchInterval: 60000, // Revalidar cada minuto
  });
}

export function useConfigs(keys: string[]) {
  return useQuery({
    queryKey: ['configs', keys],
    queryFn: async () => {
      const res = await fetch('/api/admin/config/batch', {
        method: 'POST',
        body: JSON.stringify({ keys }),
      });
      return res.json();
    },
    staleTime: 30000,
  });
}
```

4.3 Middleware de Guardia de Configuración

```
// lib/config-guard.ts
```

```typescript
import { config } from './config';
import { redirect } from 'next/navigation';

interface ConfigRequirement {
  key: string;
  expectedValue: any;
  errorMessage?: string;
}

export async function configGuard(
  requirements: ConfigRequirement[],
  options: {
    fallbackRoute?: string;
    allowSuperAdmin?: boolean;
    userRole?: string;
  } = {}
) {
  const { fallbackRoute = '/dashboard', allowSuperAdmin = true, userRole } = options;

  // Super Admin puede saltar restricciones si allowSuperAdmin = true
  if (allowSuperAdmin && userRole === 'SUPER_ADMIN') {
    return;
  }

  for (const req of requirements) {
    const value = await config.get(req.key);
    if (value !== req.expectedValue) {
      if (req.errorMessage) {
        console.warn(`ConfigGuard bloqueó acceso: ${req.errorMessage}`);
      }
      redirect(fallbackRoute);
    }
  }
}

// Uso en Server Components:
// app/dashboard/mlm/page.tsx
export default async function MLMDashboard() {
  await configGuard([
    { key: 'mlm_enabled', expectedValue: true, errorMessage: 'MLM desactivado' },
    { key: 'wallet_enabled', expectedValue: true, errorMessage: 'Wallet desactivada' },
  ]);

  return <MLMDashboardContent />;
}
```

4.4 API Routes de Configuración

```typescript
// app/api/admin/config/route.ts
import { NextRequest, NextResponse } from 'next/server';
import { getServerSession } from 'next-auth';
```

```typescript
import { config } from '@/lib/config';
import { prisma } from '@/lib/prisma';
import { z } from 'zod';

const updateSchema = z.object({
  key: z.string(),
  value: z.any(),
  reason: z.string().optional(),
});

// GET /api/admin/config - Listar todas las configuraciones
export async function GET(req: NextRequest) {
  const session = await getServerSession();
  if (!session?.user || session.user.role !== 'SUPER_ADMIN') {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 403 });
  }

  const { searchParams } = new URL(req.url);
  const category = searchParams.get('category');

  const configs = await prisma.systemConfig.findMany({
    where: category ? { category: category as any } : undefined,
    orderBy: [{ category: 'asc' }, { displayOrder: 'asc' }],
  });

  return NextResponse.json(configs);
}

// PUT /api/admin/config - Actualizar configuración
export async function PUT(req: NextRequest) {
  const session = await getServerSession();
  if (!session?.user || session.user.role !== 'SUPER_ADMIN') {
    return NextResponse.json({ error: 'Unauthorized' }, { status: 403 });
  }

  try {
    const body = await req.json();
    const { key, value, reason } = updateSchema.parse(body);

    await config.set(key, value, session.user.id, reason);

    return NextResponse.json({ success: true, key, value });
  } catch (error: any) {
    return NextResponse.json(
      { error: error.message || 'Error actualizando configuración' },
      { status: 400 }
    );
  }
}

// POST /api/admin/config/batch - Obtener múltiples configs
```

```
export async function POST(req: NextRequest) {
  const { keys } = await req.json();
  const values = await config.getMany(keys);
  return NextResponse.json(values);
}
```

4.5 Componente React de Panel de Configuración

```
// components/admin/config-panel.tsx
'use client';

import { useState } from 'react';
import { useQuery, useMutation, useQueryClient } from '@tanstack/react-query';
import { Card, CardContent, CardHeader, CardTitle } from '@/components/ui/card';
import { Switch } from '@/components/ui/switch';
import { Input } from '@/components/ui/input';
import { Button } from '@/components/ui/button';
import { Alert, AlertDescription } from '@/components/ui/alert';
import { Tabs, TabsContent, TabsList, TabsTrigger } from '@/components/ui/tabs';
import { toast } from 'sonner';

const CATEGORIES = [
  { id: 'GENERAL', label: 'General', icon: 'Settings' },
  { id: 'AUTH', label: 'Autenticación', icon: 'Shield' },
  { id: 'MEMBERSHIP', label: 'Membresías', icon: 'Crown' },
  { id: 'MLM', label: 'Plan de Compensación', icon: 'Network' },
  { id: 'WALLET', label: 'Wallet y Puntos', icon: 'Wallet' },
  { id: 'MARKETPLACE', label: 'Marketplace', icon: 'ShoppingCart' },
  { id: 'SERVICES', label: 'Servicios', icon: 'Briefcase' },
  { id: 'SECURITY', label: 'Seguridad', icon: 'Lock' },
];

export function ConfigPanel() {
  const [activeCategory, setActiveCategory] = useState('GENERAL');
  const queryClient = useQueryClient();

  const { data: configs, isLoading } = useQuery({
    queryKey: ['configs', activeCategory],
    queryFn: async () => {
      const res = await fetch(`/api/admin/config?category=${activeCategory}`);
      return res.json();
    },
  });

  const updateMutation = useMutation({
    mutationFn: async ({ key, value, reason }: any) => {
      const res = await fetch('/api/admin/config', {
        method: 'PUT',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ key, value, reason }),
      });
```

```
      if (!res.ok) throw new Error(await res.text());
      return res.json();
    },
    onSuccess: () => {
      queryClient.invalidateQueries({ queryKey: ['configs'] });
      toast.success('Configuración actualizada');
    },
    onError: (error: any) => {
      toast.error(`Error: ${error.message}`);
    },
  });

  if (isLoading) return <div>Cargando configuraciones...</div>;

  return (
    <div className="space-y-6">
      <Tabs value={activeCategory} onValueChange={setActiveCategory}>
        <TabsList className="grid grid-cols-4 lg:grid-cols-8">
          {CATEGORIES.map(cat => (
            <TabsTrigger key={cat.id} value={cat.id}>
              {cat.label}
            </TabsTrigger>
          ))}
        </TabsList>

        {CATEGORIES.map(cat => (
          <TabsContent key={cat.id} value={cat.id}>
            <div className="grid gap-4">
              {configs?.map((config: any) => (
                <ConfigCard
                  key={config.key}
                  config={config}
                  onUpdate={(value) => updateMutation.mutate({
                    key: config.key,
                    value,
                    reason: `Actualización desde panel admin`,
                  })}
                  isUpdating={updateMutation.isPending}
                />
              ))}
            </div>
          </TabsContent>
        ))}
      </Tabs>
    </div>
  );
}

function ConfigCard({ config, onUpdate, isUpdating }: any) {
  const [value, setValue] = useState(config.value);
  const [hasChanges, setHasChanges] = useState(false);
```

```
const handleChange = (newValue: any) => {
  setValue(newValue);
  setHasChanges(true);
};

const handleSave = () => {
  onUpdate(value);
  setHasChanges(false);
};

return (
  <Card>
    <CardHeader className="pb-3">
      <CardTitle className="text-sm font-medium flex items-center justify-between">
        <span>{config.description || config.key}</span>
        <code className="text-xs bg-muted px-2 py-1 rounded">{config.key}</code>
      </CardTitle>
    </CardHeader>
    <CardContent>
      {config.dependencies?.length > 0 && (
        <Alert variant="default" className="mb-3 text-xs">
          <AlertDescription>
            Requiere: {config.dependencies.join(', ')}
          </AlertDescription>
        </Alert>
      )}

      <div className="flex items-center gap-4">
        {config.type === 'BOOLEAN' && (
          <Switch
            checked={value === 'true'}
            onCheckedChange={(checked) => handleChange(String(checked))}
            disabled={isUpdating}
          />
        )}

        {(config.type === 'NUMBER' || config.type === 'DECIMAL' || config.type === 'STRING') && (
          <Input
            type={config.type === 'NUMBER' ? 'number' : 'text'}
            value={value}
            onChange={(e) => handleChange(e.target.value)}
            disabled={isUpdating}
            className="max-w-xs"
          />
        )}

        {hasChanges && (
          <Button size="sm" onClick={handleSave} disabled={isUpdating}>
            Guardar
          </Button>
```

```
          )}
        </div>

        {(config.minValue || config.maxValue) && (
          <p className="text-xs text-muted-foreground mt-2">
            Rango: {config.minValue || '∞'} - {config.maxValue || '∞'}
          </p>
        )}
      </CardContent>
    </Card>
  );
}
```

---

5. INTEGRACIÓN CON MÓDULOS EXISTENTES

5.1 Adaptación de Flujos de Negocio

Cada módulo debe leer su configuración al inicio de cada operación:

```
// Ejemplo: Cálculo de regalías
async function calculateRoyalties(orderId: string) {
  // Leer configuraciones relevantes
  const [
    mlmEnabled,
    royaltyEnabled,
    royaltyPercentage,
    royaltyLevels,
    levelPercentages,
    compressionEnabled,
  ] = await Promise.all([
    config.get('mlm_enabled'),
    config.get('mlm_royalty_enabled'),
    config.get('mlm_royalty_percentage'),
    config.get('mlm_royalty_levels'),
    config.getMany([
      'mlm_royalty_l1_pct',
      'mlm_royalty_l2_pct',
      'mlm_royalty_l3_pct',
      'mlm_royalty_l4_pct',
      'mlm_royalty_l5_pct',
      'mlm_royalty_l6_pct',
      'mlm_royalty_l7_pct',
      'mlm_royalty_l8_pct',
    ]),
    config.get('mlm_compression_enabled'),
  ]);

  // Validar que MLM y regalías estén activos
  if (!mlmEnabled || !royaltyEnabled) {
    console.log('MLM o regalías desactivados, saltando cálculo');
```

```
    return [];
  }

  // Obtener orden y calcular margen
  const order = await prisma.order.findUnique({ where: { id: orderId } });
  const margin = order.totalAmount; // Simplificado

  // Calcular pool de regalías
  const pool = margin * (royaltyPercentage / 100);

  // Obtener árbol genealógico
  const tree = await getGenealogyTree(order.userId, royaltyLevels);

  // Aplicar compresión si está activa
  const activeTree = compressionEnabled
    ? applyCompression(tree)
    : tree;

  // Distribuir regalías
  const commissions = [];
  for (let level = 1; level <= Math.min(activeTree.length, royaltyLevels); level++) {
    const user = activeTree[level - 1];
    if (!user) break;

    const percentage = levelPercentages[`mlm_royalty_l${level}_pct`] || 0;
    const amount = pool * (percentage / 100);

    commissions.push({
      userId: user.id,
      orderId,
      level,
      percentage,
      amount,
      type: 'ROYALTY',
    });
  }

  return commissions;
}
```

5.2 Migración desde v4.0 a v5.0

```
// scripts/migrate-to-v5.ts
import { prisma } from '@/lib/prisma';
import { defaultConfigs } from '@/prisma/seeds/config';

async function migrate() {
  console.log('Iniciando migración a v5.0...');

  // 1. Crear tabla SystemConfig si no existe (Prisma migrate)
```

```
  // 2. Insertar seeds de configuraciones
  for (const cfg of defaultConfigs) {
    await prisma.systemConfig.upsert({
      where: { key: cfg.key },
      update: {
        value: cfg.value,
        type: cfg.type,
        category: cfg.category,
        description: cfg.description,
        editableBy: cfg.editableBy,
        dependencies: cfg.dependencies || [],
        minValue: cfg.minValue ? new Decimal(cfg.minValue) : null,
        maxValue: cfg.maxValue ? new Decimal(cfg.maxValue) : null,
      },
      create: {
        key: cfg.key,
        value: cfg.value,
        type: cfg.type,
        category: cfg.category,
        description: cfg.description,
        editableBy: cfg.editableBy,
        dependencies: cfg.dependencies || [],
        minValue: cfg.minValue ? new Decimal(cfg.minValue) : null,
        maxValue: cfg.maxValue ? new Decimal(cfg.maxValue) : null,
      },
    });
  }

  // 3. Actualizar precios de membresía según nueva configuración
  // Preferente: $39, Pionero: $129

  // 4. Crear categoría de productos de membresía si no existe
  const membershipCategory = await prisma.category.upsert({
    where: { slug: 'productos-membresia' },
    update: {},
    create: {
      name: 'Productos de Membresía',
      slug: 'productos-membresia',
      type: 'PRODUCT',
      isMembershipCategory: true,
    },
  });

  // 5. Actualizar config con ID de categoría
  await prisma.systemConfig.update({
    where: { key: 'membership_product_catalog_id' },
    data: { value: membershipCategory.id },
  });

  console.log('Migración completada exitosamente');
}
```

```
migrate().catch(console.error);
```

---

6. TESTING DEL CONFIGURATION ENGINE

6.1 Tests Unitarios

```typescript
// tests/config.test.ts
import { describe, it, expect, beforeEach, vi } from 'vitest';
import { ConfigManager } from '@/lib/config';
import { prisma } from '@/lib/prisma';

describe('ConfigManager', () => {
  let config: ConfigManager;

  beforeEach(() => {
    config = new ConfigManager();
  });

  it('debe obtener valor booleano correctamente', async () => {
    const value = await config.get('mlm_enabled');
    expect(typeof value).toBe('boolean');
  });

  it('debe obtener valor decimal correctamente', async () => {
    const value = await config.get('mlm_royalty_percentage');
    expect(typeof value).toBe('number');
    expect(value).toBeGreaterThanOrEqual(0);
    expect(value).toBeLessThanOrEqual(100);
  });

  it('debe validar dependencias al desactivar', async () => {
    // Intentar desactivar membership_enabled mientras MLM está activo
    await expect(
      config.set('membership_enabled', false, 'admin-id')
    ).rejects.toThrow('mlm_enabled');
  });

  it('debe validar rango numérico', async () => {
    await expect(
      config.set('mlm_royalty_percentage', 150, 'admin-id')
    ).rejects.toThrow('máximo');
  });

  it('debe guardar historial al cambiar', async () => {
    await config.set('marketplace_shipping_free_threshold', 150, 'admin-id');

    const history = await prisma.configHistory.findFirst({
      where: { key: 'marketplace_shipping_free_threshold' },
      orderBy: { createdAt: 'desc' },
```

```
    });

    expect(history).toBeDefined();
    expect(history?.newValue).toBe('150');
  });

  it('debe hacer rollback correctamente', async () => {
    const history = await prisma.configHistory.findFirst({
      where: { key: 'marketplace_shipping_free_threshold' },
      orderBy: { createdAt: 'desc' },
    });

    if (history) {
      await config.rollback(
        'marketplace_shipping_free_threshold',
        history.id,
        'admin-id'
      );

      const current = await config.get('marketplace_shipping_free_threshold');
      expect(current).toBe(Number(history.oldValue));
    }
  });
});
```

6.2 Tests de Integración

```
// tests/api/config.test.ts
describe('API /api/admin/config', () => {
  it('debe rechazar acceso sin autenticación', async () => {
    const res = await fetch('/api/admin/config');
    expect(res.status).toBe(403);
  });

  it('debe rechazar acceso con rol no admin', async () => {
    // Login como cliente
    const res = await fetch('/api/admin/config', {
      headers: { Authorization: 'Bearer client-token' },
    });
    expect(res.status).toBe(403);
  });

  it('debe permitir lectura a Super Admin', async () => {
    const res = await fetch('/api/admin/config', {
      headers: { Authorization: 'Bearer superadmin-token' },
    });
    expect(res.status).toBe(200);
    const data = await res.json();
    expect(Array.isArray(data)).toBe(true);
  });
```

```
  it('debe actualizar configuración válida', async () => {
    const res = await fetch('/api/admin/config', {
      method: 'PUT',
      headers: {
        'Content-Type': 'application/json',
        Authorization: 'Bearer superadmin-token',
      },
      body: JSON.stringify({
        key: 'marketplace_shipping_free_threshold',
        value: '150',
        reason: 'Test de integración',
      }),
    });
    expect(res.status).toBe(200);
  });
});
```

---

7. MONITOREO Y ALERTAS

7.1 Métricas del Configuration Engine

Métrica	Descripción	Alerta
Config cache hit rate	% de lecturas desde cache	$< 80\%$
Config update latency	Tiempo promedio de actualización	> 500ms
Config validation failures	Intentos de cambio inválidos	> 5/min
Dependency conflicts	Bloqueos por dependencias	Cualquiera

7.2 Alertas Automáticas

```
// lib/config-alerts.ts
export async function checkConfigHealth() {
  // Verificar que todas las configs requeridas existan
  const requiredConfigs = [
    'mlm_enabled',
    'wallet_enabled',
    'membership_enabled',
    'marketplace_enabled',
  ];

  for (const key of requiredConfigs) {
    const config = await prisma.systemConfig.findUnique({ where: { key } });
    if (!config) {
      await sendAlert(`Configuración crítica faltante: ${key}`);
    }
  }

  // Verificar consistencia de dependencias
```

```
  const configs = await prisma.systemConfig.findMany();
  for (const cfg of configs) {
    if (cfg.value === 'true') {
      for (const dep of cfg.dependencies) {
        const depConfig = await prisma.systemConfig.findUnique({ where: { key: dep } });
        if (!depConfig || depConfig.value !== 'true') {
          await sendAlert(
            `Inconsistencia: ${cfg.key} está activo pero requiere ${dep}`
          );
        }
      }
    }
  }
}
```

---

8. CONCLUSIONES

El Configuration Engine transforma SaidonClub de un sistema rígido a una plataforma verdaderamente adaptable:

1. **Sin deploys para cambios de negocio:** El Super Admin controla todo.
2. **Sin errores por configuración:** Validaciones automáticas de dependencias y rangos.
3. **Sin pérdida de trazabilidad:** Cada cambio queda auditado con rollback.
4. **Sin degradación de performance:** Cache distribuida con fallback seguro.
5. **Sin conflictos entre módulos:** Activación/desactivación controlada por dependencias.

Próximo paso: Implementar este engine ANTES que cualquier otra funcionalidad de v5.0.

Configuration Engine Technical Spec | SaidonClub v5.0 | 2026-04-22